

제9강

1) 카카오 API를 활용한 도서검색

TodoList09

1. HTTP 비동기 통신을 쉽게 해주는 `axios` 자바스크립트 라이브러리를 설치한다.
`npm install axios`
2. 카카오 개발자센터 애플리케이션을 생성한다.
<https://developers.kakao.com/>
3. 내 애플리케이션의 REST API 키를 복사한다.
4. 루트 `.env` 파일에 복사한 키를 붙여 넣은 후 서버를 다시 구동한다.
`REACT_APP_KAKAOAK=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
5. `.gitignore` 파일에 `.env`를 추가하여 github에 업로드 되지 않도록 한다.
6. `BookPage` 컴포넌트에서 검색기능과 페이지 기능을 작성한다.

[src]-[components09] BookPage.jsx

```
const BookPage = () => {
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState(null);
  const size=12;
  const query='리액트';
  const page=1;

  const callAPI = async() => {
    setLoading(true);
    const url = 'https://dapi.kakao.com/v3/search/book?target=title';
    const config = {
      headers: { 'Authorization': `KakaoAK ${process.env.REACT_APP_KAKAOAK}` },
      params: { size, query, page }
    };

    const res = await axios(url, config);
    setResponse(res.data);
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, []);

  if(loading || !response) return <h1 className='text-center my-5'>로딩중....</h1>
  const { documents, meta } = response;

  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>도서검색</h1>
      <Row>
        {documents.map(book=>
          <Col key={book.isbn} xs={6} md={4} lg={3} xl={2} className='mb-3'>
            <Card>
              <Card.Body>
                <img src={book.thumbnail} width='100%' />
              </Card.Body>
              <Card.Footer>
                <div className='text-truncate'>{book.title}</div>
              </Card.Footer>
            </Card>
          </Col>
        )}
      </Row>
    </div>
  );
}

export default BookPage;
```

2) 페이징 기능과 검색기능 추가

[src]-[components09] BookPage.jsx

```
const BookPage = () => {
  const [loading, setLoading] = useState(false);
  const [response, setResponse] = useState(null);
  const [text, setText] = useState('');

  const [search] = useSearchParams();
  const size = 12;
  const query = search.get('query') || '리액트';
  const page = parseInt(search.get('page')) || 1;
  const navi = useNavigate();

  const callAPI = async() => {
    setLoading(true);
    const url = 'https://dapi.kakao.com/v3/search/book?target=title';
    const config = {
      headers: {'Authorization': `KakaoAK ${process.env.REACT_APP_KAKAOAK}`},
      params: {size, query, page}
    }
    const res = await axios(url, config);
    console.log(res.data);
    setResponse(res.data);
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  }, [page, query]);

  const onSubmit = (e) => {
    e.preventDefault();
    navi(`/book?query=${text}`);
  }

  if(loading || !response) return <h1 className='text-center my-5'>로딩중....</h1>
  const { documents, meta } = response;
  const last = Math.ceil(meta.pageable_count/12);

  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>도서검색</h1>
      <Row className='mb-3'>
        <Col xs={6} md={5} lg={4} xl={3}>
          <Form onSubmit={onSubmit}>
            <InputGroup>
              <Form.Control value={text} onChange={(e)=>setText(e.target.value)} />
              <Button type='submit'>검색</Button>
            </InputGroup>
          </Form>
        </Col>
      </Row>
      <Row>
        {documents.map(book=>
          ...
        )}
      </Row>
      <div className='text-center my-3'>
        <Button disabled={page===1} onClick={()=>navi(`/book?page=${page-1}&query=${query}`)}>이전</Button>
        <span className='mx-3'>{page}/{last}</span>
        <Button disabled={page===last} onClick={()=>navi(`/book?page=${page+1}&query=${query}`)}>다음</Button>
      </div>
    </div>
  );
}
```

3) 페이징 기능과 검색기능 분리

[src]-[components09]-[common] SearchForm.jsx

```
const SearchForm = ({ query }) => {
  const { pathname } = useLocation();
  const navi=useNavigate();
  const textRef = useRef(null);
  const [text, setText] = useState(query);

  const onSubmit = (e) => {
    e.preventDefault();
    navi(`${pathname}?query=${text} &page=1`);
  }
  useEffect(() => {
    textRef.current.focus();
  }, []);

  return (
    <Row className='mb-2'>
      <Col xs={6} md={5} lg={4} xl={3}>
        <Form onSubmit={onSubmit}>
          <InputGroup>
            <Form.Control ref={textRef} value={text} onChange={(e) => setText(e.target.value)} placeholder='검색어' />
            <Button type='submit'>검색</Button>
          </InputGroup>
        </Form>
      </Col>
    </Row>
  )
}
```

[src]-[components09]-[common] PagingButton.jsx

```
const PagingButton = ({ page, last, query }) => {
  const navi = useNavigate();
  const { pathname } = useLocation();

  return (
    <div className='text-center mt-3'>
      <Button onClick={() => navi(`${pathname}?page=${page-1} &query=${query}`)} disabled={page===1}>이전</Button>
      <span className='mx-3'>{page} / {last}</span>
      <Button onClick={() => navi(`${pathname}?page=${page+1} &query=${query}`)} disabled={page===last}>다음</Button>
    </div>
  )
}
```

[src]-[components09] BookPage.jsx

```
import SearchForm from '../common/SearchForm';
import PagingButton from '../common/PagingButton';

const BookPage = () => {
  return (
    <div className='my-5'>
      <SearchForm query={query} />
      <Row>
        <Col key={book.isbn} xs={6} md={4} lg={3} xl={2} className='mb-3'>
          ...
        </Col>
      </Row>
      { last > 1 && <PagingButton page={page} last={last} query={query} /> }
      { documents.length === 0 && <h3 className='text-center my-5'>검색 도서가 없습니다.</h3> }
    </div>
  )
}
```